FX-ENCODER++: EXTRACTING INSTRUMENT-WISE AUDIO EFFECTS REPRESENTATIONS FROM MIXTURES

***Yen-Tung Yeh**[1] **Junghyun Koo**[2] **Marco A. Martínez-Ramírez**[2]
Wei-Hsiang Liao[2] **Yi-Hsuan Yang**[1] **Yuki Mitsufuji**[2,3]
[1] National Taiwan University, Taipei, Taiwan,
[2] Sony AI, Tokyo, Japan, [3] Sony Group Corporation, Tokyo, Japan
f12942179@ntu.edu.tw

ABSTRACT

General-purpose audio representations have proven effective across diverse music information retrieval applications, yet their utility in intelligent music production remains limited by insufficient understanding of audio effects (Fx). Although previous approaches have emphasized audio effects analysis at the mixture level, this focus falls short for tasks demanding instrument-wise audio effects understanding, such as automatic mixing. In this work, we present **Fx-Encoder++**, a novel model designed to extract instrument-wise audio effects representations from music mixtures. Our approach leverages a contrastive learning framework and introduces an "extractor" mechanism that, when provided with instrument queries (audio or text), transforms mixture-level audio effects embeddings into instrument-wise audio effects embeddings. We evaluated our model across retrieval and audio effects parameter matching tasks, testing its performance across a diverse range of instruments. The results demonstrate that Fx-Encoder++ outperforms previous approaches at mixture level and show a novel ability to extract effects representation instrument-wise, addressing a critical capability gap in intelligent music production systems.

1. INTRODUCTION

Recent advances in deep learning have enabled significant progress in general-purpose audio representations [1–5], which have proven effective across diverse applications. However, they inadequately capture the nuanced characteristics of audio effects processing [6–8], since they prioritize semantic content recognition over subtle audio effects or timbral transformations. This shortcoming particularly affects specialized applications that require a precise understanding of audio effects configuration and their perceptual impact across different musical contexts.

A key domain affected by this representation gap is intelligent music production, the emerging field of mixing and mastering automation aiming to develop AI systems capable of professional-quality audio engineering [9]. Applications in this domain, including automatic mixing [10, 11], audio effects style transfer [6, 12, 13], and mixing style transfer [14, 15], all require specialized representations that model effects at two distinct levels: how they shape the overall sound of a complete mixture ("mixture level") and how they transform individual instruments within that mixture ("instrument level").

Previous approaches to audio effects representation can be categorized into two groups: those analyzing effects at the entire musical mixes [12, 15] and those examining effects on isolated single instruments [16, 17]. Although both approaches operate at what we consider the "mixture level", they fail to extract instrument-specific effect characteristics from complex mixtures. To our knowledge, FX-Encoder [15] is the only existing work that encodes audio effects information from individual instrument stems. However, they focus only on modeling the aggregate result, rather than identifying how effects have been applied to each instrument. This limitation restricts applications such as automatic mixing, where a precise understanding of the instrument-wise processing is essential.

Extracting instrument-specific audio effects representations from mixtures presents two distinct challenges. First, a straightforward approach might involve applying source separation algorithms [18, 19] to isolate individual instruments before analyzing their effects. This method is limited to separation artifacts, such as missing high frequencies, transient smearing [20], and imperfect signal reconstruction. Those artifacts will distort effect characteristics, making the separate-then-analyze pipeline unreliable. Second, to obtain accurate effect parameters, we require both the processed track (wet audio) and its unprocessed counterpart (dry audio) for reverse engineering [21, 22]. This necessitates not only perfect source separation to extract the processed track from the mixture but also access to the original unprocessed audio, which is rarely available.

To address these limitations, we propose **Fx-Encoder++**, a novel model designed to extract instrument-specific audio effects representations from complete music

*Work done during an internship at Sony AI

© Y-T. Yeh, J. Koo, M. Martínez-Ramírez, W-H. Liao, Y-H Yang, and Y. Mitsufuji. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** Y-T. Yeh, J. Koo, M. Martínez-Ramírez, W-H. Liao, Y-H Yang, and Y. Mitsufuji, "Fx-Encoder++: Extracting Instrument-Wise Audio Effects Representations from Mixtures", in *Proc. of the 26th Int. Society for Music Information Retrieval Conf.,* Daejeon, South Korea, 2025.

mixtures. Our model employs a contrastive learning framework based on SimCLR [23], which learns representations by maximizing agreement between different augmented views of the same data. Following FX-Encoder [15], we implement several crucial design elements to ensure effective learning for audio effects including: i) Fx-Normalization [24] to normalize inherent effects in source audio; ii) consistent instrumentation composition, to ensure each mixture in the entire batch is constructed with the same instrument combination, thereby isolating effect-related features; and iii) systematic audio effects manipulation procedures. The core of our contribution is an "extractor" mechanism that transforms mixture-level embeddings into instrument-specific embeddings when provided with instrument queries. This extractor leverages a pretrained CLAP encoder [3] to support both audio and text queries.

In our experiments, we evaluate Fx-Encoder++ against both general-purpose audio representations (VGGish [1], PANN [2], and CLAP [3]) and specialized audio effects encoders (FX-Encoder [15] and AFx-Rep [6]). Building upon the evaluation framework of AFx-Rep [6], we introduce a comprehensive benchmark for audio effects retrieval using real-world multitrack recordings from MUSDB [25] and MedleyDB [26]. We quantitatively assess performance across two dimensions: effects complexity (single vs. multiple cascaded effects) and instrument generalization. Results demonstrate that Fx-Encoder++ consistently outperforms existing methods in mixture-level effects extraction tasks. Furthermore, our approach enables instrument-specific effects extraction, addressing a gap in previous work where effects could only be analyzed at the mixture level.

2. RELATED WORKS

2.1 General-purpose Audio Representation

General-purpose audio representations have emerged to support various downstream tasks. VGGish [1] and PANN [2] employ CNNs trained on AudioSet [27] for audio classification and pattern recognition. More recently, CLAP [3] uses a transformer-based architecture with contrastive learning to align audio and text, enabling applications in text-to-music generation [28, 29] and audio separation [30, 31]. Neural audio compression models [4, 32, 33] represent another category, utilizing VAEs to reconstruct perceptual features while minimizing bitrate. Despite their success across various tasks, these representations show limited sensitivity to audio effects transformations [6–8]. This limitation stems from training objectives prioritizing semantic content recognition or cross-modal alignment rather than preserving the subtle timbral modifications introduced by audio effects, a capability essential for intelligent music production systems.

2.2 Audio Effects Representation

Audio effects representation research has evolved from implicit to explicit approaches. Early works incorporated effects awareness indirectly through applications like reverb parameter estimation [34], automatic mixing [35], differentiable effects [36], and neural style transfer [12, 14]. Dedicated representations emerged with FX-Encoder [15], which used contrastive learning to disentangle effects characteristics from content, enabling mixing style transfer. Similarly, Tone Embedding [16] and OpenAMP [17] focused specifically on guitar tones but was limited to isolated guitar recordings. Recently, AFx-Rep [6], a classification-based model designed for inference-time effects optimization. Its training objective is to classify which single effect is applied between two given audio clips and to further predict which preset that effect belongs to. Despite these advances, a limitation persists: existing models operate either at mixture level or on isolated instruments, but cannot extract instrument-wise effects information from complete mixes. Table 1 compares audio effects representation approaches. "Extraction Level" indicates the source from which models extract effects representations: "Isolated Inst." for single-instrument capability, "Mixture" for complete music mixes. Fx-Encoder++ uniquely extracts at both mixture and instrument-wise levels. The "Query" column shows whether the model supports conditional extraction based on instrument queries, a distinctive feature of our approach.

Model	Training Method	Audio Type	Extraction Level	Query
FX-Encoder [15]	Contrastive	Mixture	Mixture	-
Tone Emb. [16]	Contrastive	Guitar	Isolated Inst.	-
OpenAMP [17]	Contrastive	Guitar	Isolated Inst.	-
AFx-Rep [6]	Classification	Mixture	Mixture	-
Fx-Encoder++ (ours)	Contrastive	Mixture	Mixture & Inst.	✓

Table 1. Comparison of audio effect representation models. Fx-Encoder++ uniquely extracts instrument-specific effects directly from mixtures, unlike previous approaches limited to whole-mix effects or isolated instruments.

3. METHOD

Our goal is to develop an encoder, denoted as $\mathcal{E}(\mathbf{x}, \mathbf{q})$, that encodes an audio effects embedding from music mixtures $\mathbf{x}$. When conditioned with an instrument query $\mathbf{q}$, the encoder extracts effects representations specific to that instrument within the mixture; without conditioning (i.e., $\mathbf{q} = \emptyset$), the encoder produces representations that characterize the complete mixture. We formulate a contrastive objective for learning mixture-level representations following FX-Encoder [15], then extend it with a mechanism to extract instrument-specific effects information directly from mixtures, as shown in Figure 1.

3.1 Contrastive Objective for Audio Effects

Following FX-Encoder [15], we employed a SimCLR-based [23] contrastive objective to learn audio effects representations by maximizing the agreement between different audio contents processed with identical effects. The

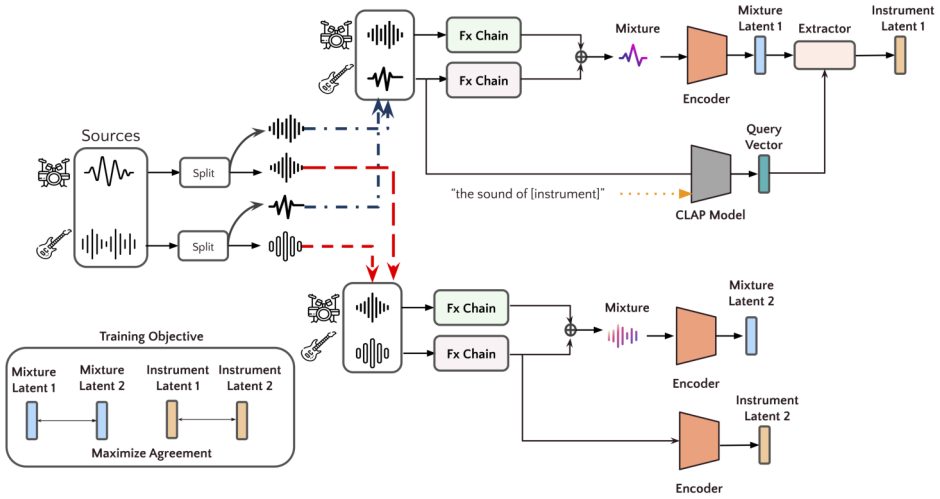


Figure 1. Our data preparation pipeline creates training pairs by splitting source tracks ($n = 2$ in this example) into segments, applying consistent effect chains, and combining them into mixtures with identical effect configurations but distinct musical content. The architecture consists of an encoder that processes mixtures to produce mixture-level embeddings, and an extractor mechanism that transforms these embeddings into instrument-specific representations using CLAP-derived queries. During inference, we can optionally provide text query to extract the specific instruments.

contrastive loss for a positive pair (i, j) can be formulated as:

$$\ell_{i,j}^{\text{mixture}} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathcal{I}_{[k\neq i]} \exp(\text{sim}(z_i, z_k)/\tau)} \quad (1)$$

where $\mathcal{I}_{[k\neq i]} \in \{0, 1\}$ is an indicator function evaluating to 1 if $k \neq i$, τ denotes a temperature parameter, and embeddings $z_i = \mathcal{E}(M_i)$ and $z_j = \mathcal{E}(M_j)$ are encoder outputs from mixtures M_i and M_j. These mixtures are generated detailed as follows:

- *Fx-Normalization [24].* Directly applying identical audio effects to different audio clips is insufficient for our contrastive objective due to inherent effects already present in each recording. When applying effects f to clips x_a and x_b with underlying effects f_a and f_b, we obtain distinct transformations $f \circ f_a$ and $f \circ f_b$ despite using the same effects f, contradicting our goal of clustering samples with the same effects in embedding space. To overcome this limitation, we employed Fx-Normalization data preprocessing method [24], the technique normalized audio characteristics by neutralizing existing effects, ensuring subsequently applied effects make equal contributions across all samples.
- *Consistent Instrument Composition.* We ensure all data samples within each training batch share identical instrument composition, preventing the model from taking shortcuts by distinguishing between instrument combinations (e.g., "drums+bass" versus "guitar+vocals") rather than effects characteristics. By controlling instrumentation across both positive and negative pairs, we force the model to focus on subtle effects features instead of more easily distinguishable instrument timbres. This is crucial for SimCLR, where numerous negative samples could otherwise lead to exploitation of simpler patterns based on instrument content [37].
- *Audio Effects Manipulation.* We create mixture pairs with identical effects processing but different musical content through a systematic approach: (1) generate k distinct audio effects chains by randomly sampling configurations (order, number, types, and parameters), from our effects pool (equalizer, delay, distortion, stereo imager, compression, limiter, reverb, and gain); (2) randomly select k instruments from the normalized pool; (3) split each instrument track into two segments with different musical content; (4) apply the same effects chain E_j to both segments of instrument j; (5) create two mixtures—M_1 and M_2—by combining all first and second segments respectively, with random loudness normalization (-18 to -14 dB LUFS) for individual track and consistent -18 dB LUFS for final mixtures. This creates positive pairs with different content but identical effects processing for contrastive learning. We also employ Fx probability scheduling [15] to prevent the model from focusing only on easily distinguishable effects.
- *Hand-Crafted Hard Negative Samples.* Hard negative samples are crucial for effective contrastive learning [38]. We explicitly construct hard negatives by applying different effects chains to identical source material, creating samples with same musical content but different effects styles. We ensure variation through both structural differences (altering effect processor types, counts, and ordering) and independent parameter sampling. This approach forces the model to focus specifically on effects-based rather than content-based features.

3.2 Learnable Extractor

A critical design requirement is producing both mixture-level and instrument-specific effects representations with high fidelity. We incorporate instrument queries from a pretrained CLAP encoder [3], allowing audio queries during training and text queries during inference [31] for intu-

itive control. While we could condition the encoder using methods like FiLM [39], this creates a challenge: the appropriate query vector for extracting the "global" mixture effects. A zero vector lacks semantic meaning, while using the mixture itself as a query would force the model to simultaneously analyze and extract from the same information stream. Our architecture therefore addresses dual requirements: extracting instrument-specific components while maintaining mixture-level representation.

Inspired from [40], our *extractor* mechanism resolves this by maintaining separate paths for mixture-level effects and instrument-specific extraction. The base encoder learns effects representations from mixtures, while the extractor transforms these based on instrument queries (audio or text). This two-stage design creates a principled interface between effects representation and instrument content representation, enabling targeted extraction of effects characteristics for specific instruments.

To facilitate the transformation of mixture effects embeddings into instrument-specific effects embeddings, we reformulate the standard contrastive loss. Rather than operating solely on mixture pairs, we develop an instrument-aware contrastive objective based on triplets (Q, M_i, M_j), where Q represents the instrument query, and M_i and M_j denote music mixtures with identical effects configurations. To make this approach compatible with mixture-level training, we construct an *extractor* mechanism that extracts instrument-specific effect embeddings from mixture-level effects embeddings using instrument queries. Formally, we define the extractor as:

$$z_i^m = \texttt{extractor}(Q_i^m, \mathcal{E}(M_i)) \tag{2}$$

where z_i^m represents the instrument-specific effects embedding, $Q_i^m \in \mathbb{R}^D$ is the query vector for the m-th instrument in mixture M_i (obtained from a pretrained CLAP encoder), and $\mathcal{E}(M_i)$ is the mixture-level audio effects embedding. The extractor is implemented as a multilayer perceptron that learns to selectively attend to effects-related features associated with the queried instrument. The instrument-aware contrastive loss is then formulated as:

$$\ell_{i,j}^{\text{inst}} = -\log \frac{\exp(\text{sim}(z_i^m, z_j^m)/\tau)}{\sum_{k=1}^{2N} \mathcal{I}_{[k \neq i]} \exp(\text{sim}(z_i^m, z_k^m)/\tau)} \tag{3}$$

In this formulation, positive pairs consist of embeddings from the same instrument type across different mixtures processed with identical effects, while negative pairs include the same instrument with different effects processing, extending Equation (1) to the instrument level. Our final training objective combines both mixture-level and instrument-level contrastive losses:

$$\mathcal{L} = \lambda_{\text{mix}} \cdot \ell_{i,j}^{\text{mixture}} + \lambda_{\text{inst}} \cdot \ell_{i,j}^{\text{inst}} \tag{4}$$

where $\lambda_{\text{mix}} + \lambda_{\text{inst}} = 1.0$. This dual-objective approach enables our model to simultaneously learn effects representations at both mixture and instrument-specific levels. We implement a curriculum learning strategy by starting with $\lambda_{\text{mix}} = 1.0$ and linearly introducing the instrument-wise objective over training step. This is necessary because instrument-wise gradients lack meaningful guidance until the model has established reasonably accurate mixture-level representations. Our final loss weighting uses $\lambda_{\text{mix}} = 0.8$ and $\lambda_{\text{inst}} = 0.2$, deliberately prioritizing mixture-level learning. This weighting reflects a critical insight: robust mixture-level effects representations form the foundation for successful instrument-specific extraction, as the extractor directly operates on these mixture-level embeddings to isolate individual components.

3.3 Model Architecture and Training Details

Audio Processing Pipeline. For effects normalization, we follow [24] but exclude reverb normalization. We implement audio processors using `dasp-pytorch` [41] and `torchcomp` [42], incorporating seven effects types (equalizer, distortion, multiband-compressor, limiter, gain, stereo imager, delay). Reverberation uses convolution-based processing with in-house impulse responses, while using `pyloudnorm` for loudness normalization [43].

Model Components. The encoder $\mathcal{E}$ is implemented using the PANN architecture [2], following established practices in audio effects processing [6]. Input audio is preprocessed by computing log-melspectrograms with a window size of 2048 samples and hop size of 512 samples. We clip magnitude values between -80 and 40 dB before scaling the spectrograms to the range $[-1, 1]$. Each input segment is 10-seconds long to adequately capture audio effects characteristics. The *extractor* mechanism employs a 3-layer MLP with hidden dimension 128 and LeakyReLU activation (slope 0.1). For instrument queries, we leverage the CLAP model [3] to produce embeddings, applying a high dropout rate (0.75 to 0.95) during training to improve generalization and bridging the modality gap between text-audio encoders in CLAP, following the approach in [31].

Training Procedure We train our model using the Adam optimizer [44] with β_1 = 0.99 and β_2 = 0.9, employing a warm-up schedule [45] followed by cosine learning rate decay from a base rate of $1e-4$. We set contrastive temperature to 0.1 and train on 2 NVIDIA H100 GPUs with a batch size of 192. For our experiments, we employ the MoisesDB dataset [46], which contains 240 tracks with 11 stems across 12 genres, totaling over 14 hours of multitrack data. It should be noted that tracks from the MoisesDB dataset [46] are "wet," meaning they have already been processed with audio effects. To enhance model's robustness across varying mixture complexities, we randomly selected between one to four instruments for constructing each mixture during training. [1]

4. EVALUATION METHOD

4.1 Audio Effects Retrieval

To evaluate our model, we conduct audio effects retrieval experiments using a controllable effects pipeline

[1] `https://github.com/SonyResearch/Fx-Encoder_PlusPlus`

Type	Model	MUSDB18 [25]																	
		Drums				Bass				Vocals				Other				Mixture		
		R@1	R@5	R@10	L_d	R@1	R@5	R@10	L_d	R@1	R@5	R@10	L_d	R@1	R@5	R@10	L_d	R@1	R@5	R@10
GP	CLAP	1.5	3.9	5.9	1.38	1.4	3.7	6.0	1.39	0.7	3.0	4.7	1.67	0.3	1.8	3.4	1.40	1.6	5.2	8.7
	PANN	0.8	2.2	3.7	1.75	0.8	2.9	4.6	1.59	0.3	1.8	3.1	1.75	0.3	0.9	2.3	1.54	1.2	3.8	6.5
	VGGish	2.0	5.7	8.5	**1.13**	1.8	4.7	6.7	1.28	1.1	3.7	5.8	1.65	0.6	2.6	4.3	1.39	1.4	5.1	7.8
Fx	FX-Encoder	12.1	25.9	35.1	1.33	7.3	17.8	24.9	1.39	6.1	16.0	22.8	1.92	5.8	15.4	22.5	**1.37**	12.3	26.6	34.5
	AFx-Rep	20.8	34.8	40.5	1.15	13.1	24.9	31.8	1.18	11.5	23.1	29.3	**1.42**	10.9	21.9	27.1	1.40	16.7	30.2	36.9
	Fx-Encoder++	**26.1**	**40.7**	**46.5**	1.16	**22.1**	**35.9**	**42.0**	**1.13**	**14.2**	**25.4**	**31.9**	1.58	**17.4**	**30.2**	**37.1**	1.38	**19.4**	**33.5**	**40.5**

Type	Model	MedleyDB [26]																	
		Mandolin				Alto Saxophone				Horn				Trumpet				Mixture		
		R@1	R@5	R@10	L_d	R@1	R@5	R@10	L_d	R@1	R@5	R@10	L_d	R@1	R@5	R@10	L_d	R@1	R@5	R@10
GP	CLAP	1.4	3.5	5.8	1.24	1.5	4.2	6.9	1.43	1.9	4.5	6.1	3.1	1.36	1.2	4.5	1.42	0.8	3.4	5.4
	PANN	0.6	2.5	4.9	1.24	0.8	2.4	4.3	1.53	1.4	3.4	4.9	1.33	0.9	2.2	3.3	1.42	0.7	2.4	4.9
	VGGish	1.7	4.4	7.5	1.26	2.0	5.4	8.1	1.44	2.8	5.7	7.5	1.30	1.5	3.6	5.1	1.33	0.9	2.6	3.8
Fx	FX-Encoder	2.3	6.3	10.1	1.27	2.2	6.5	10.0	1.38	3.4	8.5	11.9	1.33	1.7	4.8	6.8	1.36	0.8	3.0	5.0
	AFx-Rep	15.8	25.2	30.8	**1.15**	12.7	21.3	26.1	1.37	12.2	19.9	24.4	**1.17**	8.0	14.3	17.7	1.40	4.2	10.1	13.6
	Fx-Encoder++	**17.5**	**27.8**	**34.1**	1.20	**18.6**	**29.4**	**35.4**	**1.33**	**13.5**	**21.6**	**25.9**	1.24	**9.1**	**16.1**	**19.8**	**1.30**	**5.6**	**12.4**	**17.3**

Table 2. Audio effects retrieval performance (R@K values in percentages, higher is better) on isolated instruments and mixtures. The L_d column indicates performance in audio effects parameter matching tasks (lower is better). GP: general purpose audio representations. Fx: audio effects specific representations.

Type	Model	MUSDB18 [25]											
		Target Instrument (Oracle)				*USS*(m) / *MSS*(m)				$\mathcal{E}(\mathbf{x}, \mathbf{q_{audio}})$ / $\mathcal{E}(\mathbf{x}, \mathbf{q_{text}})$			
		R@1	R@5	R@10	mAP@10	R@1	R@5	R@10	mAP@10	R@1	R@5	R@10	mAP@10
GP	CLAP	1.0	3.1	5.0	2.0	0.3 / 0.3	1.2 / 1.5	2.4 / 2.8	0.8 / 0.8	-	-	-	-
	PANN	0.6	2.0	3.4	1.2	0.3 / 0.3	1.1 / 1.4	2.3 / 2.4	0.7 / 0.8	-	-	-	-
	VGGish	1.4	4.2	6.3	2.6	0.3 / 0.5	1.3 / 1.7	2.7 / 3.0	0.7 / 1.1	-	-	-	-
Fx	FX-Encoder	7.8	18.8	26.3	12.6	0.6 / 2.0	2.4 / 7.0	4.1 / 11.5	1.4 / 4.3	-	-	-	-
	AFx-Rep	14.0	26.2	32.2	19.3	**2.1** / 4.6	6.0 / 10.8	8.8 / 15.0	3.8 / 7.3	-	-	-	-
	Fx-Encoder++	**19.9**	**33.0**	**39.4**	**25.6**	**2.1 / 7.1**	**6.4 / 15.4**	**9.9 / 20.5**	**4.0 / 10.8**	**3.0 / 3.0**	**8.1 / 8.1**	11.7 / **12.2**	5.2 / **5.3**

Type	Model	MedleyDB [26]											
		Target Instrument (Oracle)				*USS*(m)				$\mathcal{E}(\mathbf{x}, \mathbf{q_{audio}})$ / $\mathcal{E}(\mathbf{x}, \mathbf{q_{text}})$			
		R@1	R@5	R@10	mAP@10	R@1	R@5	R@10	mAP@10	R@1	R@5	R@10	mAP@10
GP	CLAP	0.9	2.6	4.4	1.7	0.3	1.5	2.8	0.9	-	-	-	-
	PANN	2.0	4.8	7.0	3.2	0.3	1.3	2.5	0.8	-	-	-	-
	VGGish	1.5	3.8	5.8	2.6	0.4	1.4	2.5	0.8	-	-	-	-
Fx	FX-Encoder	2.4	6.5	9.7	4.2	0.6	2.2	3.9	1.4	-	-	-	-
	AFx-Rep	12.1	20.2	24.8	15.6	**1.8**	**4.7**	**7.2**	**3.1**	-	-	-	-
	Fx-Encoder++	**14.7**	**23.7**	**28.8**	**18.7**	1.7	4.6	7.0	3.0	1.6 / **1.9**	5.0 / **5.7**	7.6 / **8.7**	3.1 / **3.6**

Table 3. Audio effects retrieval performance (R@K values and mAP in percentages) on instrument-wise extraction.

with MUSDB [25] and MedleyDB [26]. Our evaluation builds upon AFx-Rep [6] and makes further improvements in three dimensions, including: (1) expanding small retrieval pools (previously limited to only 20 samples), (2) implementing effects normalization (addressing the issue of inherent effects in audio), and (3) extending performance metrics (beyond the previously reported accuracy).

Evaluation Framework. Our retrieval task evaluates how accurately models identify correct effects configurations from 500 candidates when presented with processed audio. Given audio query, our goal is to retrieve the target audio with same effects configurations but different musical contents. The task uses cosine similarity in the embedding space between queries and candidate pools, following the evaluation methodology established in CLAP [3]. We report Recall@K (R@K) in percentages, applying effects to evaluation datasets using the procedure described in Section 3.1. We construct separate evaluation sets from MUSDB [25] (vocals, drums, bass, other) and from MedleyDB [26] (mandolin, saxophone, horn, trumpet) to test generalization across instrument diversity. We further assess performance across effects complexity by varying the number of effects applied to each track from 1 to 8.

Baselines. We compare against general-purpose representations (VGGish [1], PANN [2], CLAP [3]) and specialized effects representations (FX-Encoder [15], AFx-Rep [6]), excluding guitar-specific models [16, 17] unsuitable for multi-instrument evaluation.

Retrieval Scenarios. We evaluate using two complementary scenarios: (1) *Mixture-level Retrieval*, testing effects identification in complete mixtures or isolated recordings [6, 15]; and (2) *Instrument-wise Retrieval*, evaluating our novel extraction capability with both ground-truth isolated tracks and directly from mixtures. For baseline models lacking instrument-specific capabilities, we implement a two-stage approach using universal sound separa-

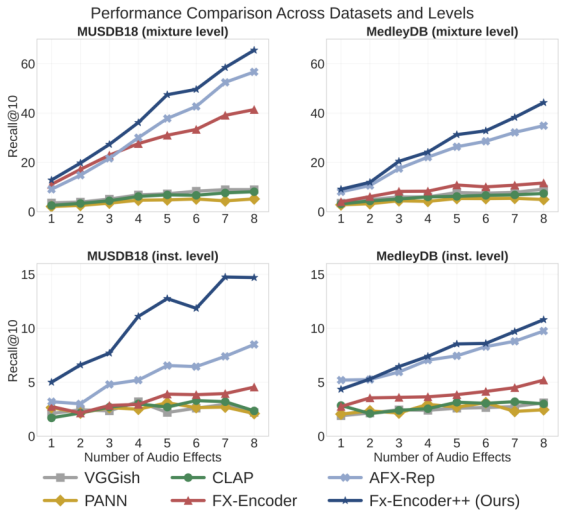


Figure 2. Performance comparison across datasets, mixture and instrument-level, and effects complexity.

tion [18] for arbitrary instrument types and Hybrid Demucs [47] for MUSDB18's four standard tracks (vocals, bass, drums, other). In contrast, Fx-Encoder++ directly extracts instrument-specific embeddings without requiring separation. We support both audio and text queries through our CLAP encoder, using the prompt template: "*this is the sound of {target instrument}*" [31] for text queries.

4.2 Matching of Audio Effects Parameters

We evaluate downstream applications through effects parameters matching following [6], extracting effects representations from reference audio to optimize differentiable effect chain parameters applied to dry input while minimizing a multi-resolution STFT Loss [48]. Our evaluation uses audio triplets (clean, reference, target) where input/target share identical content with different applied audio effects, while reference/target have different content but identical effects. We synthesize 100 samples per instrument from MUSDB [25] and MedleyDB [26] datasets, applying seven sequential effects (EQ, multiband compressor, stereo imager, gain, distortion, delay, limiter). We compare against the same baselines used in our retrieval experiments.

5. RESULTS

5.1 Audio Effects Retrieval

Mixture-level Retrieval. Table 2 shows Fx-Encoder++ substantially outperforms both general-purpose models and other effect-specific models on MUSDB dataset. Our model particularly excels with "drums" and "bass" . However, performance on "vocals" is comparatively lower than other instruments (drops approximately 10%), likely due to the high timbral variation of vocals in each song (e.g. male vs. female singers). This pattern is also observed in the "other" and "mixture" categories, which similarly exhibit high timbral variety. All models show decreased performance on the MedleyDB dataset, suggesting that performance is limited by the frequency of instrument exposure during training. Despite this challenge, Fx-Encoder++ maintains its performance advantage across datasets.

Instrument-wise Retrieval. Table 3 presents retrieval results across three protocols: (1) Target Instrument (Oracle): using ground-truth isolated tracks; (2) *USS*(m) / *MSS*(m): applying universal [18] or music source separation [47]; and (3) $\mathcal{E}(\mathbf{x}, \mathbf{q_{audio}})/\mathcal{E}(\mathbf{x}, \mathbf{q_{text}})$: our extractor applied directly to mixtures. For MUSDB dataset, our model substantially outperforms general-purpose models on target instruments. USS-based methods suffer significant performance drops (9.9% R@10), while MSS performs better (20.5% R@10) but is limited to only four predefined stems. Notably, even when using high-quality source separation, we observe a clear gap between the "Target Instrument" and "*MSS*(m)", (approximately 19% lower), indicating that separation artifacts may distort the effects characteristics. Our extractor outperforms *USS(m)* without requiring an external source separation model, with text queries perform slightly better than audio queries, probably because they provide a more generalized instrument space compared the q_{audio}, as the query audio has different content and Fx than the target. For the MedleyDB dataset, performance degrades across all protocols, highlighting challenges in generalizing to instruments observed less frequently during training.

Number of Effects. Figure 2 demonstrates our model's superior performance across varying effect counts. Performance improves with more effects for all models, indicating complex effect chains create more distinctive signatures than single effect, which may be confused with natural instrument timbres. For MUSDB dataset, the performance gap widens with increasing effects complexity. While similar trends appear for MedleyDB dataset, reduced performance indicates challenges in generalizing to novel timbral characteristics.

5.2 Matching of Audio Effects Parameters

Table 2 shows varied matching performance (L_d) across instruments. Unlike retrieval tasks, no model consistently outperforms across all scenarios. For MUSDB dataset, different models excel in different categories (VGGish on drums, Fx-Encoder++ on bass, AFx-Rep on vocals), with general-purpose VGGish surprisingly outperforming specialized models in several cases. For MedleyDB dataset, AFx-Rep and Fx-Encoder++ achieve the lowest reconstruction losses. These findings suggest that retrieval and effect parameter matching tasks benefit from different representational characteristics.

6. CONCLUSION

We introduced **Fx-Encoder++**, the first model that extracts instrument-wise audio effects information from music mixtures. Our approach outperforms existing methods in audio effects retrieval at the mixture level while enabling instrument-wise effects embedding extraction. Limitations include reduced effect parameter matching performance, and difficulties with single effect understanding. Future work should address these challenges by improving retrieval-transformation bridging, enhancing single effect understanding for applications like VA modeling [49–51].

7. ACKNOWLEDGEMENTS

Yen-Tung thanks National Science and Technology Council for supporting his PhD study.

8. REFERENCES

[1] S. Hershey, S. Chaudhuri, D. P. Ellis, J. F. Gemmeke, A. Jansen, R. C. Moore, M. Plakal, D. Platt, R. A. Saurous, B. Seybold *et al.*, "CNN architectures for large-scale audio classification," in *Proc. International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2017.

[2] Q. Kong, Y. Cao, T. Iqbal, Y. Wang, W. Wang, and M. D. Plumbley, "PANNs: Large-scale pretrained audio neural networks for audio pattern recognition," *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 28, 2020.

[3] Y. Wu, K. Chen, T. Zhang, Y. Hui, T. Berg-Kirkpatrick, and S. Dubnov, "Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation," in *Proc. International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2023.

[4] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, "High fidelity neural audio compression," *Trans. Machine Learning Research*, 2023.

[5] S. Chen, Y. Wu, C. Wang, S. Liu, D. Tompkins, Z. Chen, and F. Wei, "Beats: Audio pre-training with acoustic tokenizers," in *Proc. ICML*, 2022.

[6] C. J. Steinmetz, S. Singh, M. Comunità, I. Ibnyahya, S. Yuan, E. Benetos, and J. D. Reiss, "ST-ITO: Controlling audio effects for style transfer with inference-time optimization," in *Proc. International Society for Music Information Retrieval (ISMIR)*, 2024.

[7] S. H. Hawley and C. J. Steinmetz, "Leveraging neural representations for audio manipulation," in *154th Convention of the Audio Engineering Society*, 2023.

[8] A. Chu, P. O'Reilly, J. Barnett, and B. Pardo, "Text2fx: Harnessing clap embeddings for text-guided audio effects," in *Proc. International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2025.

[9] B. De Man, R. Stables, and J. D. Reiss, *Intelligent music production*. Focal Press, 2019.

[10] C. J. Steinmetz, J. Pons, S. Pascual, and J. Serrà, "Automatic multitrack mixing with a differentiable mixing console of neural audio effects," in *Proc. International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2021.

[11] M. Martinez Ramirez, D. Stoller, and D. Moffat, "A deep learning approach to intelligent drum mixing with the wave-u-net," *Journal of the Audio Engineering Society*, vol. 69, 2021.

[12] C. J. Steinmetz, N. J. Bryan, and J. D. Reiss, "Style transfer of audio effects with differentiable signal processing," *J. Audio Eng. Soc*, vol. 70, 2022.

[13] J. Koo, S. Paik, and K. Lee, "End-to-end music remastering system using self-supervised and adversarial training," in *Proc. International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2022.

[14] S. S. Vanka, C. Steinmetz, J.-B. Rolland, J. Reiss, and G. Fazekas, "Diff-MST: Differentiable mixing style transfer," in *Proc. International Society for Music Information Retrieval (ISMIR)*, 2024.

[15] J. Koo, M. A. Martínez-Ramírez, W.-H. Liao, S. Uhlich, K. Lee, and Y. Mitsufuji, "Music mixing style transfer: A contrastive learning approach to disentangle audio effects," in *Proc. International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2023.

[16] Y.-H. Chen, Y.-T. Yeh, Y.-C. Cheng, J.-T. Wu, Y.-H. Ho, J.-S. R. Jang, and Y.-H. Yang, "Towards zero-shot amplifier modeling: One-to-many amplifier modeling via tone embedding control," in *Proc. International Society for Music Information Retrieval (ISMIR)*, 2024.

[17] A. Wright, A. Carson, and L. Juvela, "Open-Amp: Synthetic data framework for audio effect foundation models," in *Proc. International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2025.

[18] Q. Kong, K. Chen, H. Liu, X. Du, T. Berg-Kirkpatrick, S. Dubnov, and M. D. Plumbley, "Universal source separation with weakly labelled data," *arXiv preprint arXiv:2305.07447*, 2023.

[19] A. Défossez, N. Usunier, L. Bottou, and F. Bach, "Music source separation in the waveform domain," *arXiv preprint arXiv:1911.13254*, 2019.

[20] N. Schaffer, B. Cogan, E. Manilow, M. Morrison, P. Seetharaman, and B. Pardo, "Music separation enhancement with generative modeling," in *Proc. International Society for Music Information Retrieval (ISMIR)*, 2022.

[21] J. T. Colonel and J. Reiss, "Reverse engineering of a recording mix with differentiable digital signal processing," *The Journal of the Acoustical Society of America*, vol. 150, no. 1, pp. 608–619, 2021.

[22] S. Lee, M. A. Martínez-Ramírez, W.-H. Liao, S. Uhlich, G. Fabbro, K. Lee, and Y. Mitsufuji, "Searching for music mixing graphs: A pruning approach," in *27th International Conference on Digital Audio Effects (DAFx)*, 2024.

[23] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. ICML*, 2020.

[24] M. A. Martínez-Ramírez, W.-H. Liao, G. Fabbro, S. Uhlich, C. Nagashima, and Y. Mitsufuji, "Automatic music mixing with deep learning and out-of-domain data," in *Proc. International Society for Music Information Retrieval (ISMIR)*, 2022.

[25] Z. Rafii, A. Liutkus, F.-R. Stöter, S. I. Mimilakis, and R. Bittner, "MUSDB18-HQ - an uncompressed version of MUSDB18," Aug. 2019. [Online]. Available: https://doi.org/10.5281/zenodo.3338373

[26] R. M. Bittner, J. Salamon, M. Tierney, M. Mauch, C. Cannam, and J. P. Bello, "MedleyDB: A multitrack dataset for annotation-intensive mir research." in *Proc. International Society for Music Information Retrieval (ISMIR)*, 2014.

[27] J. F. Gemmeke, D. P. Ellis, D. Freedman, A. Jansen, W. Lawrence, R. C. Moore, M. Plakal, and M. Ritter, "Audio Set: An ontology and human-labeled dataset for audio events," in *Proc. International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2017.

[28] H. Liu, Y. Yuan, X. Liu, X. Mei, Q. Kong, Q. Tian, Y. Wang, W. Wang, Y. Wang, and M. D. Plumbley, "AudioLDM 2: Learning holistic audio generation with self-supervised pretraining," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2024.

[29] J. Copet, F. Kreuk, I. Gat, T. Remez, D. Kant, G. Synnaeve, Y. Adi, and A. Défossez, "Simple and controllable music generation," in *Proc. NeurIPS*, 2023.

[30] X. Liu, Q. Kong, Y. Zhao, H. Liu, Y. Yuan, Y. Liu, R. Xia, Y. Wang, M. D. Plumbley, and W. Wang, "Separate anything you describe," *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, 2024.

[31] K. Saijo, J. Ebbers, F. G. Germain, S. Khurana, G. Wiehern, and J. L. Roux, "Leveraging audio-only data for text-queried target sound extraction," in *Proc. International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2025.

[32] R. Kumar, P. Seetharaman, A. Luebs, I. Kumar, and K. Kumar, "High-fidelity audio compression with improved rvqgan," in *Proc. NeurIPS*, 2023.

[33] M. Pasini, S. Lattner, and G. Fazekas, "Music2latent: Consistency autoencoders for latent audio compression," in *Proc. International Society for Music Information Retrieval (ISMIR)*, 2024.

[34] J. Koo, S. Paik, and K. Lee, "Reverb conversion of mixed vocal tracks using an end-to-end convolutional deep neural network," in *Proc. International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2021.

[35] B.-Y. Chen, W.-H. Hsu, W.-H. Liao, M. A. M. Ramírez, Y. Mitsufuji, and Y.-H. Yang, "Automatic dj transitions with differentiable audio effects and generative adversarial networks," in *Proc. International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2022.

[36] S. Lee, H.-S. Choi, and K. Lee, "Differentiable artificial reverberation," *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 30, 2022.

[37] J. Robinson, L. Sun, K. Yu, K. Batmanghelich, S. Jegelka, and S. Sra, "Can contrastive learning avoid shortcut solutions?" in *Proc. NeurIPS*, 2021.

[38] J. Robinson, C.-Y. Chuang, S. Sra, and S. Jegelka, "Contrastive learning with hard negative samples," in *Proc. International Conference on Learning Representations (ICLR)*, 2021.

[39] E. Perez, F. Strub, H. De Vries, V. Dumoulin, and A. Courville, "Film: Visual reasoning with a general conditioning layer," in *Proc. AAAI*, 2018.

[40] H.-Y. Chen, Z. Lai, H. Zhang, X. Wang, M. Eichner, K. You, M. Cao, B. Zhang, Y. Yang, and Z. Gan, "Contrastive localized language-image pre-training," *arXiv preprint arXiv:2410.02746*, 2024.

[41] C. J. Steinmetz, "dasp-pytorch," [Online] https://github.com/csteinmetz1/dasp-pytorch/.

[42] C.-Y. Yu, C. Mitcheltree, A. Carson, S. Bilbao, J. D. Reiss, and G. Fazekas, "Differentiable all-pole filters for time-varying audio systems," in *International Conference on Digital Audio Effects (DAFx)*, 2024.

[43] C. J. Steinmetz and J. Reiss, "pyloudnorm: A simple yet flexible loudness meter in python," in *Audio Engineering Society Convention 150*, 2021.

[44] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. International Conference on Learning Representations (ICLR)*, 2014.

[45] P. Goyal, P. Dollár, R. Girshick, P. Noordhuis, L. Wesolowski, A. Kyrola, A. Tulloch, Y. Jia, and K. He, "Accurate, Large Minibatch SGD: Training imagenet in 1 hour," *arXiv preprint arXiv:1706.02677*, 2017.

[46] I. Pereira, F. Araújo, F. Korzeniowski, and R. Vogl, "MoisesDB: A dataset for source separation beyond 4-stems," in *Proc. International Society for Music Information Retrieval (ISMIR)*, 2023, pp. 619–626.

[47] A. Défossez, "Hybrid spectrogram and waveform source separation," *arXiv preprint arXiv:2111.03600*, 2021.

[48] C. J. Steinmetz and J. D. Reiss, "auraloss: Audio focused loss functions in PyTorch," in *Digital Music Research Network One-day Workshop (DMRN+15)*, 2020.

[49] M. A. Martínez Ramírez, E. Benetos, and J. D. Reiss, "Deep learning for black-box modeling of audio effects," *Applied Sciences*, vol. 10, 2020.

[50] Y.-T. Yeh, W.-Y. Hsiao, and Y.-H. Yang, "Hyper recurrent neural network: Condition mechanisms for black-box audio effect modeling," in *International Conference on Digital Audio Effects (DAFx)*, 2024.

[51] C. J. Steinmetz and J. D. Reiss, "Efficient neural networks for real-time modeling of analog dynamic range compression," *152nd Convention of the Audio Engineering Society*, 2021.